CORE GAMEPLAY & NETWORKING

DOC ID: WC-ARCH-2026-VOL2

WildCard 게임 아키텍처 분석 보고서 (Vol. 2)

인게임 코어 프레임워크, 이중 상태 머신 및 체인 해석 엔진

프로젝트:	WildCard (Unreal Engine 5.7 Dedicated Server)	**작성 일시:**	2026-10-19
핵심 클래스:	AInGameMode, AInGameState, AInGamePlayerState, UPhaseComponent, UChainComponent	**주요 매커니즘:**	Dual State Machine, Simultaneous Main Action, LIFO Chain Stack
문서 범위:	인게임 액터 계층 구조, 6단계 페이즈 전이, 타이머 동기화, 액션 검증, 체인 공방 및 전장 미리보기 액터 풀링		

1. 인게임 코어 액터 계층 구조와 책임 분할

WildCard의 인게임 아키텍처는 언리얼 엔진의 게임플레이 프레임워크(GameMode, GameState, PlayerState, PlayerController, Pawn)를 Dedicated Server 환경의 엄격한 권한 분리 모델에 맞춰 특화 확장하였습니다.

액터 / 컴포넌트	권한(Role)	네트워크 복제	핵심 책임 및 시스템적 역할
AInGameMode	SERVER ONLY	복제 안 됨	매치 라이프사이클 중재, UPhaseComponent 소유, 시작 유닛 배치, 플레이어 패배 및 매치 종료 판정.
AInGameState	REPLICATED	전체 클라이언트	공개 매치/페이즈 상태 관리, UChainComponent 및 UWildCombatLogComponent 소유, 전역 이벤트 버스.
AInGamePlayerState	REPLICATED	소유자/전체	플레이어 자원(PlayerStatComp), 손패/덱(PlayerCardComp), 영웅 유닛 및 성유물, 비공개 행동 보관.
AInGameController	CLIENT/AUTONOMOUS	서버-클라이언트	로컬 마우스 레이캐스팅, 3D 타일 인터랙션, HUD 생성 및 서버 RPC (Server_SubmitAction) 송신 게이트웨이.
AInGamePawn	CLIENT/AUTONOMOUS	소유 클라이언트	EnhancedInput 기반 탑다운 전술 카메라(SpringArm/Camera)의 이동, 회전, 줌 제어.

1.1. AInGameState: 전역 상태 머신 및 이벤트 버스 (Event Bus)

AInGameState는 모든 클라이언트와 동기화되는 중앙 허브입니다. UI, 유닛, 이펙트 매니저 간의 강한 결합(Tight Coupling)을 제거하기 위해 다양한 **다이내믹 멀티캐스트 델리게이트(Dynamic Multicast Delegates)**를 이벤트 버스로 제공합니다.

Source/WildCard/InGame/Game/InGameState.h

C++ / UE 5.7

```cpp
UCLASS()
class WILDCARD_API AInGameState : public AGameState
{
    GENERATED_BODY()

public:
    virtual void GetLifetimeReplicatedProps(TArray<FLifetimeProperty>& OutLifetimeProps) const override;

    // --- 전역 이벤트 버스 (Event Bus) ---
    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnTurnStartedDelegate OnTurnStartedDelegate;

    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnTurnEndedDelegate OnTurnEndedDelegate;

    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnPhaseChangeDelegate OnPhaseChangedDelegate;

    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnTimerChangedDelegate OnTimerChangedDelegate;

    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnMatchStateChangeDelegate OnMatchStateChangedDelegate;

    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnChainPromptChangedDelegate OnChainPromptChangedDelegate;

    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnMatchEndedDelegate OnMatchEndedDelegate;

    UPROPERTY(BlueprintAssignable, Category = "Event Bus")
    FOnVisualCueReceivedDelegate OnVisualCueReceivedDelegate;

protected:
    UPROPERTY(ReplicatedUsing = OnRep_CurrentMatchState, BlueprintReadOnly)
    EWildMatchState CurrentMatchState;

    UPROPERTY(ReplicatedUsing = OnRep_CurrentPhase, BlueprintReadOnly)
    EWildCardPhase CurrentPhase;

    UPROPERTY(ReplicatedUsing = OnRep_PhaseRemainTime, BlueprintReadOnly)
    int32 PhaseRemainTime;

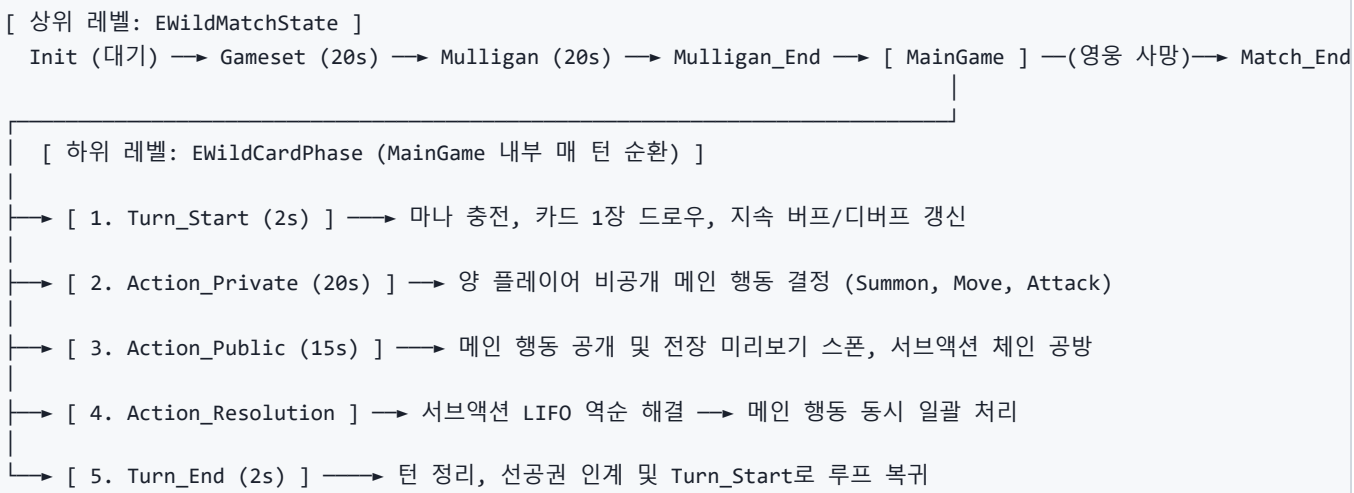
    UPROPERTY(VisibleAnywhere, BlueprintReadOnly)
    TObjectPtr<UChainComponent> ChainComponent;

    UPROPERTY(VisibleAnywhere, BlueprintReadOnly)
    TObjectPtr<UWildCombatLogComponent> CombatLogComponent;
};
```

2. 이중 상태 머신 (Dual State Machine) 아키텍처

WildCard의 게임 진행은 상위의 **매치 상태 머신(Match State Machine)**과 하위의 **턴 페이즈 루프(Turn Phase Loop)**의 이중 구조로 체계화되어 있습니다.

■ WildCard 이중 상태 머신 생명주기 다이어그램



매치 상태 (EWildMatchState)	기본 제한 시간	상태 진입 시 주요 처리 및 네트워크 동작
Init	무제한	플레이어 접속 대기 및 타일맵 초기화(InitMap).
Gameset	20초	최초 선공권 추첨, 성유물 및 전술스킬 비공개 선택 UI 표시.
Mulligan	20초	시작 손패 5장 지급, 교체할 카드 선택(Server_SubmitMulligan).
Mulligan_End	즉시/애니메이션	카드 덱 재셔플, 교체 카드 드로우 및 클라이언트 손패 확정 동기화.
MainGame	턴 루프 지속	턴 사이클 가동 (SetGamePhase(Turn_Start)).
Match_End	무제한	승자/패자 판정, 승리/패배 공지 위젯 표시, 결과 화면 진입.

3. UPhaseComponent 페이즈 제어 및 시간 동기화 엔진

UPhaseComponent는 AInGameMode에 부착되어 서버 권한에서 턴의 흐름을 통제합니다. 글로벌 1초 타이머 틱(OnGlobalTimerTick)을 통해 남은 시간을 계산하고 GameState->SetPhaseRemainTime을 호출하여 모든 클라이언트의 UI 타이머를 일괄 동기화합니다.

3.1. 비공개 행동 제출 및 검증 (ProcessPrivateActionReady)

Action_Private 페이즈 동안 플레이어는 비공개 행동(유닛 소환, 이동, 공격 등)을 서버로 전송합니다. 서버는 제출된 행동의 무결성을 검증하고 PendingMainAction에 비밀리에 보관합니다. 양 플레이어가 모두 결정을 완료하거나 제한 시간(20초)이 만료되면 즉시 다음 페이즈(Action_Public)로 전환됩니다.

Source/WildCard/InGame/Battle/PhaseComponent.cpp C++ / UE 5.7

```cpp
void UPhaseComponent::ProcessPrivateActionReady(APlayerController* PC, const FWildActionData& ActionData
{
    if (GetOwnerRole() != ROLE_Authority || !PC) return;

    AInGamePlayerState* PS = PC->GetPlayerState<AInGamePlayerState>();
    AInGameState* GameState = GetWorld()->GetGameState<AInGameState>();
    if (!PS || !GameState) return;

    // 1. 유효성 검증 및 살균(Sanitization)
    FWildActionData FinalAction = ValidatePrivateAction(PS, ActionData);

    // 2. 비공개 메인 행동 슬롯에 저장 (클라이언트에 복제되지 않음)
    PS->SetPendingMainAction(FinalAction);
    PS->bIsActionReady = true;
    ActionReadyCount++;

    UE_LOG(LogWildPhase, Log, TEXT("[Phase] Player %d 비공개 행동 제출 완료 (현재 완료: %d/2)"), PS->GetPla

    // 3. 양 플레이어 모두 제출 완료 시 즉시 대기 시간을 종료하고 Action_Public으로 전이
    if (ActionReadyCount >= 2)
    {
        AdvanceFromPrivateActionPhase();
    }
}

void UPhaseComponent::AdvanceFromPrivateActionPhase()
{
    GetWorld()->GetTimerManager().ClearTimer(PhaseTimerHandle);
    ActionReadyCount = 0;

    AInGameState* GameState = GetWorld()->GetGameState<AInGameState>();
    UChainComponent* ChainComp = GameState ? GameState->FindComponentByClass<UChainComponent>() : nullpt

    // 양 플레이어의 비공개 행동을 체인 컴포넌트로 전달하여 공개 목록으로 등록
    for (APlayerState* State : GameState->PlayerArray)
    {
        if (AInGamePlayerState* PS = Cast<AInGamePlayerState>(State))
        {
            if (ChainComp)
            {
                ChainComp->DeclareAction(PS->GetPendingMainAction());
            }
            PS->ClearPendingMainAction();
            PS->bIsActionReady = false;
        }
    }

    // 전장 미리보기 생성 및 Action_Public 페이즈 개시
    SetGamePhase(EWildCardPhase::Action_Public);
}
```

3.2. 체인 공방 및 패스(Pass) 판정 (ProcessPublicPass)

Action_Public 페이즈에서는 양 플레이어가 번갈아가며 공개된 행동에 대응(서브액션)할 기회를 갖습니다. 한 플레이어가 패스하면 상대방에게 체인 권한(Prompt)이 이전되며, **연속 2회 패스(ConsecutivePassCount >= 2)**가 발생하면 체인 공방이 마감되고 Action_Resolution 페이즈로 전환됩니다.

4. UChainComponent 체인 해석 및 동시 처리 엔진

4.1. 메인 행동과 서브 행동의 분리 아키텍처

WildCard의 체인 엔진은 행동의 성격에 따라 처리 파이프라인을 완전히 분리합니다.

- **메인 행동 (Main Action):** 턴마다 단 1회 수행하는 핵심 전술(유닛 소환, 타일 이동, 공격). 체인 스택에 쌓이지 않고 PendingResolutionMainActions에 보관된 후 모든 서브액션 해결 후 **완전 동시(Simultaneous)**에 적용됩니다.
- **서브 행동 (Sub Action):** 공개된 메인 행동에 대응하여 발동하는 즉발 스킬 및 함정. ActionStack에 LIFO(후입선출) 방식으로 쌓이며, 체인 마감 시 **역순(Reverse Order)**으로 해결됩니다.

4.2. 역순 체인 해결 및 최종 동시 처리 (ResolveChain)

Source/WildCard/InGame/Battle/ChainComponent.cpp C++ / UE 5.7

```cpp
void UChainComponent::ResolveChain()
{
    AInGameState* GameState = GetWorld()->GetGameState<AInGameState>();

    // 1단계: 서브 액션 체인 스택 (LIFO) 역순 해결
    UE_LOG(LogWildChain, Log, TEXT("[Server] === 체인 스택 역순 처리 시작 (총 스택: %d) ==="), ActionStack.
    while (ActionStack.Num() > 0)
    {
        FWildActionData SubAction = ActionStack.Pop();
        ExecuteAction(SubAction); // 데미지, 버프, 주문 효과 발동
    }
    ActionCount = 0;

    // 2단계: 유발 효과(Trigger Effects) 및 미들 페이즈 검사
    ProcessMiddlePhase();
}

void UChainComponent::ExecuteMainActionsSimultaneously()
{
    UE_LOG(LogWildChain, Warning, TEXT("[Server] ★★★ 메인 행동 일괄 동시 처리 (총 행동: %d) ★★★"), Per

    // 보관된 양 플레이어의 메인 행동을 동시에 실행
    for (const FWildActionData& Action : PendingResolutionMainActions)
    {
        ExecuteAction(Action);
    }

    bool bActionOccurred = (PendingResolutionMainActions.Num() > 0);
    PendingResolutionMainActions.Empty();
    ClearPublicMainActions(); // 전장 미리보기 액터 반환

    // 3단계: 페이즈 컴포넌트에 액션 종료 통보 -> Turn_End 전이
    if (AInGameMode* GM = Cast<AInGameMode>(GetWorld()->GetAuthGameMode()))
    {
        if (GM->PhaseComponent)
        {
            GM->PhaseComponent->CheckActionPhaseEnd(bActionOccurred);
        }
    }
}
```

4.3. 전장 미리보기 시스템 및 액터 오브젝트 풀링 (Object Pooling)

Action_Public 진입 시 클라이언트 전장에는 각 플레이어가 선언한 이동/공격 경로를 표시하는 **미리보기 화살표(AWildActionPreviewArrow)**와 소환될 유닛의 반투명 **고스트 유닛(AWildPreviewGhostUnit)**이 스폰됩니다.

매 턴마다 액터를 동적으로 생성(Spawn)하고 파괴(Destroy)하면 언리얼 엔진의 가비지 컬렉션(GC) 및 메모리 단편화 부하가 발생하므로, UChainComponent는 **오브젝트 풀(PooledPreviewArrows, PooledGhostUnits)**을 구축하여 재사용하도록 설계되었습니다.

Source/WildCard/InGame/Battle/ChainComponent.cpp C++ / UE 5.7

```cpp
AWildActionPreviewArrow* UChainComponent::GetOrCreatePooledArrow()
{
    for (AWildActionPreviewArrow* Arrow : PooledPreviewArrows)
    {
        if (IsValid(Arrow) && !Arrow->IsActorTickEnabled() && Arrow->IsHidden())
        {
            Arrow->SetActorHiddenInGame(false);
            Arrow->SetActorTickEnabled(true);
            return Arrow;
        }
    }

    // 풀에 사용 가능한 인스턴스가 없으면 신규 생성 후 풀에 추가
    FActorSpawnParameters Params;
    Params.Owner = GetOwner();
    AWildActionPreviewArrow* NewArrow = GetWorld()->SpawnActor<AWildActionPreviewArrow>(AWildActionPrevi
    PooledPreviewArrows.Add(NewArrow);
    return NewArrow;
}

void UChainComponent::ClearPublicPreviews()
{
    // 액터를 Destroy하지 않고 숨김 처리하여 풀로 반환
    for (AActor* Actor : ActivePreviewActors)
    {
        if (IsValid(Actor))
        {
            Actor->SetActorHiddenInGame(true);
            Actor->SetActorTickEnabled(false);
        }
    }
    ActivePreviewActors.Empty();
}
```

성능 최적화 효과:
액터 풀링 도입을 통해 턴 전환 시점의 액터 스폰/소멸 오버헤드를 0ms로 억제하였으며, Dedicated Server 및 저사양 클라이언트에서도 프레임 드랍 없이 부드러운 턴 진행을 보장합니다.

5. 결론 및 요약

Volume 2에서는 Dedicated Server 기반 인게임 프레임워크의 코어 액터 관계, 이중 상태 머신(MatchState/CardPhase), PhaseComponent의 정밀한 타이머/검증 로직, 그리고 ChainComponent의 LIFO 서브액션 체인 및 메인 행동 동시 처리 구조를 규명하였습니다.

차기 볼륨 안내:
이어지는 **Volume 3**에서는 카드 구역(Zone)과 결정론적 셔플, 성유물(Artifact) 발동 체계, 전술스킬(Tactical Skill)의 전략 패턴, 헥사곤 축 좌표계(Q, R) 및 3D 카드 빌보드 유닛 아키텍처를 분석합니다.